UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

MASTER THESIS

Phan Thanh Binh

ICT.2440058

Major: Data Mining for IoT

Enhanced Evaluation Protocols for Few-Shot Open-Set Keyword Spotting Systems

---

Supervisor: Dr. Tran Hoang Tung

Academic Year: 2024-2025

Declaration

I, Phan Thanh Binh, hereby declare that my thesis represents my own work after doing an internship at USTH ICT Lab.

I have read the University's internship guidelines. If plagiarism appears in my thesis, I accept responsibility for conducting the procedures in accordance with the University's Committee.

Hanoi, August 2025

Signature

Phan Thanh Binh

Acknowledgements

First and foremost, I would like to express my deepest gratitude to my supervisor, Dr. Tran Hoang Tung, for his enthusiastic guidance throughout the 3 months of internship.

I would also like to extend my thanks to Dr. Tran Giang Son for providing access to the ICTLab server, which made the computational aspects of this project become efficient.

Lastly, I want to show my appreciation to my family for their constant love and support.

Contents

Abstract **i**

1 Introduction **1**

1.1 Context and Motivation 1

1.2 Objectives 2

1.3 Report Structure 3

2 Background **4**

2.1 Prototypical Network 4

2.1.1 The Core Idea 4

2.1.2 Training Protocol with Episodic Batching 5

2.2 Few-Shot Open-Set KWS System Using Prototypical Networks 7

2.2.1 System Workflow 7

2.2.2 Audio Processing Pipeline 8

2.2.3 Conducted Experiments 13

2.3 Dataset 15

2.3.1 GSC 15

2.3.2 MSWC 17

2.4 Evaluation Metrics 18

2.4.1 Confusion Matrix 19

2.4.2 AUC 19

2.4.3 FAR and FRR 20

3 Methodology **21**

3.1 Analysis of the Original Approach 21
3.1.1 Data Utilization Strategy 21
3.1.2 Prediction Score Computation 22
3.1.3 Performance Metrics 22
3.2 Proposed Improvements 23
3.2.1 Direct Distance-Based Classification 23
3.2.2 Mean Detection Error Tradeoff Curves 24
3.2.3 Enhanced Data Utilization Framework 24
3.2.4 Multi-Lingual Spoken Words Corpus Integration 25
3.3 Experimental Design 26
3.3.1 Direct Distance vs. Probability-Based Classification 26
3.3.2 Data Utilization Strategy Comparison 26
3.3.3 Few-Shot Performance Scaling 27

4 Results & Discussion **28**
4.1 Direct Distance vs. Probability-Based Classification 28
4.2 Data Utilization Strategy Comparison 29
4.3 Few-Shot Performance Scaling 30

5 Conclusions & Future works **32**
5.1 Future Works 33

References **34**

Abstract

This study presents an enhanced evaluation framework for few-shot open-set keyword spotting (KWS) systems, addressing critical limitations in existing testing protocols. Building upon the unified pipeline framework introduced by Rusci et al., we propose three fundamental improvements to evaluation methodology: (1) direct L2 distance-based classification that eliminates artificial probability normalization constraints, (2) comprehensive mean Detection Error Tradeoff (DET) curve analysis that captures per-class performance characteristics, and (3) enhanced data utilization strategies incorporating randomized word selection and expanded vocabulary through the Multilingual Spoken Words Corpus (MSWC).

Our experimental validation demonstrates that direct distance-based classification achieves superior performance compared to probability-based approaches, with improvements of 0.01 in AUC, 0.05 in accuracy at 5% false acceptance rate, and 0.04 reduction in false rejection rate. The proposed evaluation framework reveals significant performance variations across different data utilization strategies, with fixed word partitions showing artificially optimistic results compared to randomized selections that better simulate real-world deployment scenarios. Few-shot performance scaling analysis indicates diminishing returns beyond 5-shot enrollment, with performance improvements plateauing at higher sample counts.

The enhanced testing protocol provides more rigorous assessment of model generalizability and offers clearer insights into system behavior through intuitive threshold interpretation. These contributions establish a more reliable foundation for evaluating few-shot open-set KWS systems and facilitate better comparison of different approaches in the literature. The methodology is validated across multiple experimental configurations, demonstrating its effectiveness and practical applicability for advancing keyword spotting research.

Keywords: *Keyword Spotting, On-Device Customization, Metric Learning, Prototypical Network, Open-set Classification, Evaluation Protocol.*

1. Introduction

1.1 Context and Motivation

Keyword Spotting (KWS) represents a fundamental challenge in speech processing where systems must automatically detect the occurrence of predefined target keywords or phrases within continuous audio streams. This technology serves as the backbone for numerous real-world applications, including voice-activated assistants such as Amazon Alexa and Google Assistant, hands-free control systems in automotive environments, and smart home devices that respond to wake words like ”Hey Siri” or ”OK Google.” The everywhere presence of KWS in modern human-computer interaction underscores its critical importance in creating intuitive and accessible interfaces across diverse technological platforms.

While KWS is fundamentally a detection problem that requires localizing keywords within continuous audio streams, the practical implementation typically transforms this challenge into a real-time classification task. This transformation occurs through a windowing approach where the continuous audio stream is segmented into short, overlapping windows, typically lasting one second in duration [2]. Each window is then processed independently by a classification model that determines whether the target keyword is present within. The scope of our study is primarily focused on the quality of the classification model, leaving out concerns regarding the handling of continuous classification outputs.

The increasing demand for personalized and adaptable voice interfaces has revealed a critical limitation in traditional keyword spotting (KWS) systems: their inability to rapidly customize target keywords without extensive retraining. This challenge becomes particularly acute in scenarios where users need to define custom wake words or where applications must adapt to domain-specific terminology. To address this limitation, researchers have increasingly explored few-shot learning approaches, with Prototypical Networks emerging as a particularly promising framework. Prototypical Networks allow KWS systems to learn new keywords from just a handful of examples by constructing prototype repre-

sentations in an embedding space, enabling rapid adaptation to new vocabulary without the computational overhead of full model retraining. An essential feature for any KWS system is open-set classification, namely the ability to respond with ”I don’t know” when encountering audio segments that do not contain any of the target keywords. This open-set recognition capability is crucial for preventing false positives and ensuring that systems respond only to valid keyword utterances rather than irrelevant speech.

Recognizing that the current literature on few-shot open-set KWS remains highly fragmented and lacks clear design guidelines for on-device customization, Rusci et al. [4] proposed a unified evaluation framework for such problem. Furthermore, they introduced a novel training strategy that uses different datasets for the training and testing phases, a design choice that more accurately reflects real-world deployment conditions and enables a more rigorous assessment of model generalization.

1.2 Objectives

Building upon the unified pipeline framework introduced by Rusci et al. [4], this study critically examines their testing phase protocol, with a particular focus on data utilization strategies, score computation methodologies, and evaluation metrics employed in few-shot open-set keyword spotting (KWS) systems.

To address these concerns and contribute to more rigorous evaluation standards, this study aims to:

- Perform a comprehensive analysis of the existing testing protocol, assessing data selection procedures, score computation algorithms, and performance metrics, in order to identify potential limitations that may compromise the reliability and interpretability of experimental results in few-shot open-set KWS.
- Design and propose an enhanced testing protocol that mitigates the identified shortcomings, explicitly tailored to address the dual challenges of few-shot learning and open-set recognition in KWS applications.
- Conduct experimental validation of the proposed testing protocol under various configurations to demonstrate its effectiveness and practical applicability.

1.3 Report Structure

The remainder of this report is organized as follows:

- **Chapter 2**: Provides relevant background information.
- **Chapter 3**: Describes in details the methodologies used.
- **Chapter 4**: Presents and discusses the experimental results.
- **Chapter 5**: Summarizes the conclusions and outlines directions for future work.

2. Background

2.1 Prototypical Network

2.1.1 The Core Idea

The prototypical networks framework, introduced in [5], represents a foundational paradigm shift in how we approach classification problems with limited training data. To understand its significance in the context of keyword spotting, we must first appreciate the fundamental challenge it addresses: how can a system learn to classify new categories when only a handful of examples are available for each category?

The core insight of prototypical networks lies in learning a metric space where classification can be performed by computing distances to prototype representations. Rather than learning to directly predict class labels through traditional softmax classifiers, the system learns to embed input samples into a feature space where samples from the same class cluster together, while samples from different classes remain well-separated.

Think of this approach as learning to create a "representative fingerprint" for each class. During training, the network learns how to extract meaningful features from input samples and combine multiple examples of the same class into a single prototype vector that captures the essence of that class. This prototype serves as an anchor point in the learned feature space, representing what makes that particular class distinctive.

The mathematical elegance of this approach becomes apparent when we consider how classification decisions are made. Given a new query sample, the system embeds it into the same learned feature space and computes its distance to each class prototype (see figure 2.1). The classification decision is then based on the nearest prototype, following the intuitive principle that similar samples should have similar representations.

Formally, let $f_\theta : \mathbb{R}^D \to \mathbb{R}^M$ be a neural network with parameters θ that maps input samples

to an M-dimensional embedding space. For each class k with support set $S_k = \{(x_i, y_i)\}_{i=1}^{|S_k|}$ where $y_i = k$, the prototype $\mathbf{c}_k$ is computed as the mean of the embedded support samples:

$$\mathbf{c}_k = \frac{1}{|S_k|} \sum_{(x_i, y_i) \in S_k} f_\theta(x_i) \tag{2.1}$$

Given a query sample x_q, the probability of it belonging to class k is determined by a softmax over the negative squared Euclidean distances to each prototype:

$$p_\theta(y = k | x_q) = \frac{\exp(-d(f_\theta(x_q), \mathbf{c}_k))}{\sum_{k'} \exp(-d(f_\theta(x_q), \mathbf{c}_{k'}))} \tag{2.2}$$

where $d(\cdot, \cdot)$ represents the squared Euclidean distance function.

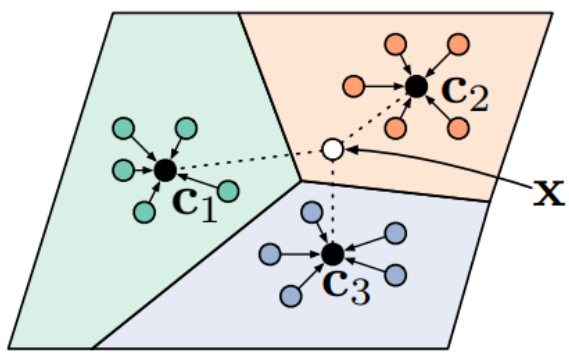

Figure 2.1: Prototypical Network illustration [5]

2.1.2 Training Protocol with Episodic Batching

The training protocol for prototypical networks fundamentally differs from conventional supervised learning approaches. Instead of processing data in traditional mini-batches where samples from all classes are mixed together, prototypical networks employ an episodic training strategy that explicitly mimics the few-shot learning scenario the system will encounter during deployment.

The episodic training paradigm is designed to create training episodes that mirror the structure of few-shot learning tasks. Each episode can be thought of as a miniature learning problem where the model must quickly adapt to classify a small set of novel classes based on only a few examples per class. This approach ensures that the feature encoder learns representations that are immediately useful for prototype-based classification, rather than

learning features optimized for a large fixed vocabulary.

During each training episode, we construct what is known as an N-way K-shot classification task. Here, N represents the number of classes to be distinguished within the episode, while K indicates the number of support examples available for each class. The episode construction process begins by randomly sampling N classes from the training dataset, ensuring these classes are different from those used in previous episodes within the same training batch.

For each selected class, we then randomly sample K examples to form the support set, along with additional query examples that will be used for computing the loss. The support examples serve the dual purpose of computing class prototypes and providing the context for classification, while the query examples represent the samples that must be classified using the learned prototypes.

The mathematical formulation of an episode can be expressed as follows. Let $\mathscr{D}_{train}$ represent our training dataset containing many classes. For each episode e, we sample a subset of classes $\mathscr{C}_e \subset \mathscr{D}_{train}$ where $|\mathscr{C}_e| = N$. For each class $c \in \mathscr{C}_e$, we sample K support examples and Q query examples, where Q is typically set to ensure sufficient gradient signal for training.

The support set for episode e is defined as:

$$\mathscr{S}_e = \bigcup_{c \in \mathscr{C}_e} \{(x_i^c, c)\}_{i=1}^{K} \tag{2.3}$$

Similarly, the query set is:

$$\mathscr{Q}_e = \bigcup_{c \in \mathscr{C}_e} \{(x_j^c, c)\}_{j=K+1}^{K+Q} \tag{2.4}$$

The training objective for each episode involves computing prototypes from the support set and then evaluating the classification accuracy on the query set. The loss function typically employs negative log-likelihood of the correct class assignments:

$$\mathscr{L}_e = -\frac{1}{N \cdot Q} \sum_{(x_q, y_q) \in \mathscr{Q}_e} \log p_\theta(y = y_q | x_q) \tag{2.5}$$

This episodic structure provides several crucial benefits for learning effective few-shot classification systems. Most importantly, it forces the network to learn feature representations that work well with prototype-based classification from the very beginning of training. Tra-

ditional approaches might learn features optimized for large-vocabulary classification that prove suboptimal when adapted to few-shot scenarios.

Furthermore, the episodic approach naturally incorporates the variability inherent in few-shot learning scenarios. By constantly changing which classes appear in each episode and which specific examples represent each class, the network learns to extract robust, generalizable features rather than memorizing specific class-example combinations. This variability is essential for developing systems that can handle the diversity of real-world deployment scenarios.

The choice of N and K during training deserves careful consideration, as it directly impacts the learned representations. Training with larger values of N encourages the network to learn more discriminative features that can distinguish between many classes simultaneously. Conversely, training with smaller values of K forces the network to extract maximally informative features from minimal data, which is crucial for few-shot performance.

2.2 Few-Shot Open-Set KWS System Using Prototypical Networks

This section presents the system design presented in [4], which demonstrates how prototypical networks can be effectively employed for on-device customization of keyword spotting systems.

2.2.1 System Workflow

The KWS system proposed by [4] follows a two-stage pipeline that separates the computationally intensive training phase from the lightweight deployment phase (figure 2.2). The overall workflow consists of the following key components:

Training Phase: A deep neural network feature encoder f_θ is trained on a large-scale source dataset (Multilingual Spoken Words Corpus) using episodic training protocols. The encoder learns to map audio inputs to a discriminative embedding space where samples from the same keyword class cluster together while different classes remain well-separated.

Enrollment Phase: During deployment, users provide a small number of enrollment samples (typically 1-10 samples) for each desired keyword. These samples are processed through the pre-trained frozen feature encoder to generate embeddings, which are then

averaged to compute class prototypes:

$$\mathbf{c}_k = \frac{1}{|S_k|} \sum_{(x_i, y_i) \in S_k} f_\theta(x_i) \tag{2.6}$$

where S_k represents the support set for class k and $\mathbf{c}_k$ is the resulting prototype.

Inference Phase: For each incoming query sample x_q, the system computes its embedding $f_\theta(x_q)$ and measures its distances to all stored prototypes. The open-set classifier then produces a probability score vector $P = \{p_i\}_{i=0}^{N}$ based on the prototype set. Here, p_0 represents the prediction score for the *unknown class*, while $p_{i \neq 0}$ corresponds to the probability of the i-th keyword. The probability of a keyword is higher when its distance score, specifically euclidean or L2 distance, is lower.

The final class prediction y is obtained as follows:

$$y = \begin{cases} \underset{i}{\operatorname{argmax}}\, p_i, & \text{if } \max_i p_i \geq \gamma \\ 0, & \text{otherwise,} \end{cases} \tag{2.7}$$

where $y = 0$ denotes the unknown class, and γ is a manually tunable threshold parameter that balances precision and recall in the classifier.

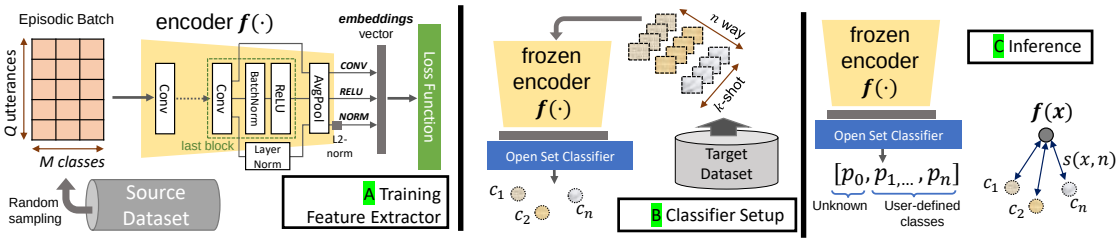

Figure 2.2: Few-shot Open-set KWS system design in [4]

2.2.2 Audio Processing Pipeline

The audio processing workflow transforms raw speech signals into discriminative embeddings through the following stages

2.2.2.1 Speech features extraction

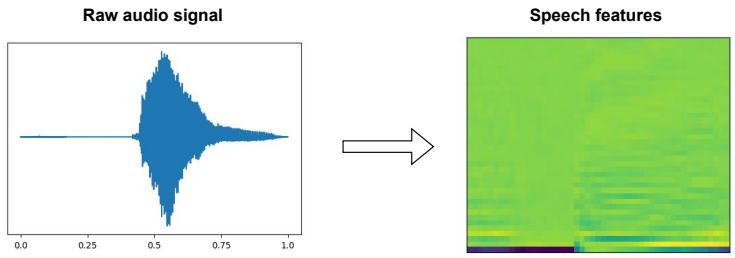

Figure 2.3: Speech feature extraction illustration

Raw audio samples are converted into 40-dimensional MFCC features using a 40ms window with 20ms stride. This creates a time-frequency representation $\mathbf{X} \in \mathbb{R}^{T \times 40}$ where T represents the temporal dimension. Mel Frequency Cepstral Coefficients (MFCC) is a method that has been popular for decades in the KWS field [2]. To summarize, MFCC is calculated by applying Discrete Cosine Transform (DCT) on the log Mel-spectrogram of the audio signal. The detailed steps and formulas are explained below.

Pre-emphasis: The first step is to enhance the audio signal by amplifying the high frequency components. This helps the details, important features of the signal become more discernible.

$$x[n] = y[n] - \alpha y[n-1]$$

where $x[n]$ is the enhanced audio signal, $y[n]$ is the original audio signal and α is typically set to 0.97

Framing and Windowing: The next step is to divide the audio signal into short, overlapping segments called frames. This segmentation step is vital because speech characteristics, like pitch and tone, change over time and are hard to capture as a whole. Therefore, combining the features of these short, stationary segments, is a more effective way to capture speech features of the whole audio signal.

$$x_m[n] = x[n + m \cdot H] \quad (0 \leq n < N, \quad 0 \leq m < M)$$

where $x_m[n]$ is the m-th frame, M is the number of frames, N is the number of samples in each frame and H is the hop size or frame shift

A common practice after obtaining the frames is to smooth them by a window function to

reduce the edge effects.

$$x_m[n] = x_m[n] \cdot w[n],$$

where $w[n] = 0.54 - 0.46 \cos \frac{2\pi n}{N-1}$.

Fourier Transform: The next step is to convert the frames from the time domain into the frequency domain by applying the Discrete Fourier Transform (DFT).

$$X[k] = \sum_{n=0}^{N-1} x_m[n] e^{-j2\pi k \cdot n/N}$$

where $X[k]$ is the DFT of the m-th frame, k is the frequency bin index.

Then, the power spectrum of the audio signal can be obtained, which describes the distribution of power over frequency bands.

$$P[k] = \frac{|X[k]|^2}{N}$$

where $P[k]$ is the power value, $|X[k]|$ is the magnitude of $X[k]$.

Mel Filtering: Following that, the power spectrum is put through a number of triangular Mel filters (typically from 20 to 40).

$$S_m = \sum_{k=f_{m-1}}^{f_{m+1}} P[k] H_m[k]$$

where S_m, $H_m[k]$, f_m are the output, the value and the frequency bin boundaries of the m-th filter respectively.

To attain the filters, the lower and upper boundaries of the relevant frequency need to be defined first. Subsequently, M values between the lower and upper boundaries are computed where M is the number of desired filters. These values, including the boundaries, are linearly spaced on the Mel-scale. To achieve this, a formula to convert from the Hz-scale to the Mel-scale is given.

$$M(f) = 2595 \cdot \log_{10}(1 + \frac{f}{700})$$

where $M(f)$, f are the frequency value on the Mel-scale and the Hz-scale respectively. This function is reversible so the conversion from the Mel-scale back to the Hz-scale can also be

easily computed. Next, the frequencies are assigned to corresponding DFT frequency bin.

$$f[i] = \lfloor (n_{dft} + 1) \cdot h[i]/sr \rfloor$$

where n_{dft} is the size of the DFT, sr is the sample rate of the audio signal and $h[i]$, $f[i]$ are the bin index and real value of the frequency respectively. Finally, the filters are calculated by the following formula.

$$H_m[k] = \begin{cases} 0 & k < f[i-1] \\ \frac{k-f[i-1]}{f[i]-f[i-1]} & f[i-1] \leq k \leq f[i] \\ \frac{f[i+1]-k}{f[i+1]-f[i]} & f[i] \leq k \leq f[i+1] \\ 0 & k > f[i+1] \end{cases}$$

Log Mel-spectrogram and MFCC The output from all the steps above for all the frames are concatenated together in the temporal order, forming a matrix with $T * F$ shape, where T is the number of frames and F is the number of Mel filters. This matrix is also known as the Mel-spectrogram of the audio signal. Lastly, Discrete Cosine Transform (DCT) operation is applied on this matrix, following logarithmic transformation, along the axis corresponding to the Mel filters coefficients.

$$C[k] = 2\beta \cdot \sum_{f=0}^{F-1} S_f \cdot \cos \frac{\pi k(2f+1)}{2F}$$

where $C[k]$ is the MFCC value and

$$\beta = \begin{cases} \sqrt{1/4F} & \text{if } k = 0 \\ \sqrt{1/2F} & \text{otherwise} \end{cases}$$

2.2.2.2 Feature Encoding

The MFCC features are processed by a deep neural network to generate embeddings for the input audio signals. The architecture employed is a Depthwise Separable Convolutional Neural Network (DSCNN), which is primarily composed of depthwise separable convolutional blocks. Depthwise separable convolution has been shown to be an efficient alternative to the standard 3D convolution operation, enabling the design of compact network architectures in the field of computer vision [7]. The core idea is to decompose a

standard 3D convolution into two lightweight operations: a depthwise convolution and a pointwise convolution, as illustrated in Figure 2.4.

Depthwise convolution The core concept is that only one single convolutional filter is applied per input channel. For an input feature map X with dimensions (H,W,C), the depthwise convolution can be mathematically expressed as

$$Y^{(c)} = X^{(c)} * K^{(c)}$$

where $Y^{(c)}$, $X^{(c)}$, $K^{(c)}$ are the output, input feature map and the convolutional filter for channel c respectively.

Pointwise convolution The core concept is that the combinations of output across channels are only carried out by 1x1 convolutional filters. For an input feature map Y with dimensions (H,W,C), the pointwise convolution can be mathematically expressed as

$$Z_k = \sum_{c=1}^{C} Y^{(c)} * W_k^{(c)}$$

where Z_k, W_k is the output feature map and the pointwise convolution filters for the k-th channel of the next layer respectively.

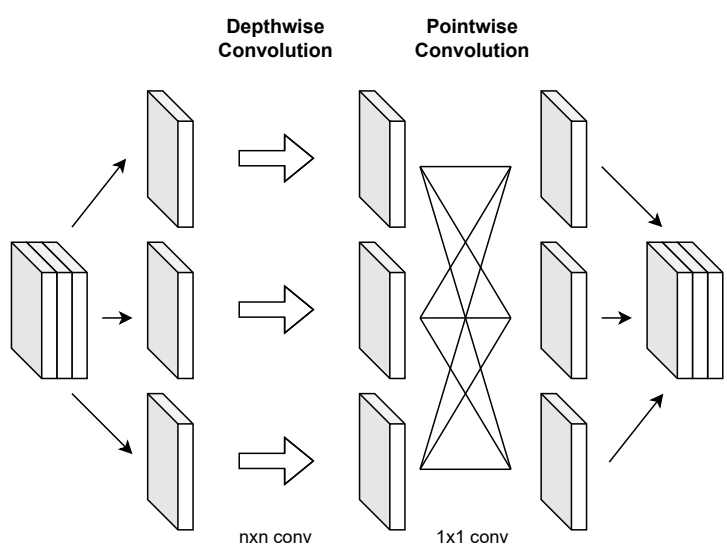

Figure 2.4: DSCNN block

2.2.3 Conducted Experiments

[4] designed a comprehensive experimental framework to evaluate the effectiveness of different feature encoder training strategies combined with various open-set classification mechanisms. Their experiments systematically compare multiple configurations across different model sizes and few-shot scenarios.

2.2.3.1 Training Loss Functions

The authors investigate three different training approaches for the feature encoder:

Prototypical Networks (PN): Traditional episodic training where 10-way episodes with 10 support and 30 query samples are constructed from 40 randomly selected classes. Training loss is computed on query samples using softmax over negative distances to prototypes.

$$\mathscr{L}_{PN} = -\frac{1}{Q \cdot M} \sum_{i=1}^{Q} \sum_{j=1}^{M} \log \frac{\exp\left(s(\mathbf{x}_{i,j}^{Q}, j)\right)}{\sum_{k=1}^{M} \exp\left(s(\mathbf{x}_{i,j}^{Q}, k)\right)} \tag{2.8}$$

where $s(\mathbf{x}, j) = -d_{L2}\big(f(\mathbf{x}), \mathbf{c}_j\big)$

Angular Prototypical (AP): A variant that replaces Euclidean distance with cosine similarity:

$$s(x, j) = w \cdot (\cos(f(x), \mathbf{c}_j) - m) + b \tag{2.9}$$

where w and b are learnable parameters and $m = 0.5$ is the margin parameter.

Triplet Loss (TL): Direct metric learning approach that optimizes the embedding space to ensure that samples from the same class are closer to each other than to samples from different classes. The triplet loss operates on triplets $\{x_i, x_i^+, x_i^-\}$ where x_i is an anchor sample, x_i^+ is a positive sample from the same class as the anchor, and x_i^- is a negative sample from a different class. The loss function enforces that the distance between anchor and positive should be smaller than the distance between anchor and negative by at least a margin m:

$$\mathscr{L}_{TL} = \frac{1}{N_t} \sum_{i=1}^{N_t} \max(0, d(x_i, x_i^+) - d(x_i, x_i^-) + m) \tag{2.10}$$

The training uses episodic batches of 20 samples × 80 classes, with negative samples randomly selected from different categories within each batch and margin $m = 0.5$. This direct

optimization of the distance metric proves particularly effective for few-shot scenarios as it explicitly learns embeddings where intra-class distances are minimized while inter-class distances are maximized.

2.2.3.2 Open-Set Classification Methods

Three open-set classification approaches are evaluated to handle unknown keywords:

Open Nearest Class Mean (openNCM): A simple extension of the standard NCM classifier that estimates an unknown class prototype $\mathbf{c}_0$ using K random samples from the target domain that do not belong to user-defined classes. Classification probabilities are computed using softmax over the $(N+1)$-sized distance vector.

OpenMAX: A statistical approach that models the distance distribution of known classes using Weibull distributions fitted from the 5 largest distances in the enrollment samples. Test sample distances are scaled by factors $w_i^{wb} \in [0,1]$, and the unknown class score is estimated as:

$$s(x, j=0) = -\sum_{i=1}^{N} (1 - w_i^{wb}) \cdot d_{L2}(f(x), \mathbf{c}_i) \tag{2.11}$$

Dummy Proto (DProto): A generator network $g(\cdot)$ jointly trained with the feature encoder to produce dummy prototypes for unknown classes. The generator creates 3 unknown prototypes, with the closest one selected at inference time. During training, 16 out of 40 classes in each episode are designated as unknown to train both the feature extractor and generator.

2.2.3.3 Experimental Configurations

The experiments employ two DSCNN variants: DSCNN-L (407k parameters) and DSCNN-S (22k parameters), each tested with three feature extraction modes (CONV, RELU, NORM). Training is conducted for 40 epochs of 400 episodes using Adam optimizer with initial learning rate 0.001, decayed by 0.1 after 20 epochs.

The evaluation framework uses the Multilingual Spoken Words Corpus (MSWC) for training, restricting data sampling to the 500 classes with the highest number of utterances after excluding target dataset categories. Training samples are augmented with additive background noises from the DEMAND dataset, applied with 0.95 probability and random SNR between 0 and 5 dB to simulate realistic acoustic conditions.

Testing is performed on the Google Speech Commands (GSC) dataset, with 10 target classes for few-shot classification and 20 classes serving as unknown categories. Performance is assessed using accuracy at 5% false acceptance rate ($ACC_5\%^+$), false rejection rate at 5% FAR ($FRR_5\%^+$), and AUROC metrics, with results averaged over 10 test repetitions.

2.2.3.4 Best Configuration and Key Findings

The experimental results reveal several important findings:

Optimal Configuration: The best performing setup consists of DSCNN-L with normalized features (NORM) trained using triplet loss and paired with openNCM classifier, achieving 76% accuracy in 10-shot scenarios while maintaining 5% false acceptance rate.

Training Loss Comparison: Triplet loss significantly outperforms both prototypical networks (PN) and angular prototypical (AP) variants, showing 8-10% accuracy improvements on DSCNN-L configurations.

Feature Normalization Impact: L2 normalization proves crucial for triplet loss effectiveness, with DSCNN-L-NORM achieving 4-5% accuracy improvements over CONV and RELU variants when trained with triplet loss.

Open-Set Classifier Performance: openNCM consistently achieves the highest accuracies across different configurations, while OpenMAX shows poor performance in few-shot scenarios due to insufficient samples for reliable statistical modeling.

These findings establish triplet loss with normalized DSCNN features and openNCM classification as the preferred configuration for few-shot open-set keyword spotting systems.

2.3 Dataset

2.3.1 GSC

Google Speech Commands [6] is an audio dataset of spoken words in English created by Google. The dataset was built to provide a publicly accessible benchmark for training and evaluate KWS systems, which was not available at the time. The utterances are contributed voluntarily via a web-based application.

The dataset includes 35 words divided into three distinct subsets. The first subset consists

of the 10 digits from 0 to 9. The second subset contains 15 useful keywords for IoT applications: ”Yes,” ”No,” ”Up,” ”Down,” ”Left,” ”Right,” ”On,” ”Off,” ”Stop,” ”Go,” ”Backward,” ”Forward,” ”Follow,” and ”Learn.” The final subset of 10 words—”Bed”, ”Bird”, ”Cat”, ”Dog”, ”Happy”, ”House”, ”Marvin”, ”Sheila”, ”Tree”, and ”Wow”—serve as tests for the ability of the KWS engine to ignore speech that does not contain triggers. In summary, there are 105829 utterances recorded by 2,618 speakers in total, with the distribution over the 35 words shown in figure 2.5. Each utterance is stored in the WAV format with the duration of 1 second (or less), at 16KHz sampling rate. The utterances for each word are split into train, dev, test sets based on a hash function applied on the file name. The implementation code in Python can be found here [1]

The dataset can be downloaded from this link [2] as a gzip-compressed tar archive, which occupies approximately 2.3 GiB of disk storage. After uncompressing, the audio files are organized into folders label with their respective word (see figure 2.6). Additionally, there is a extra folder named *_background_noise_* which contains 1-minute clips (also at 16KHz sampling rate) of various background noises.

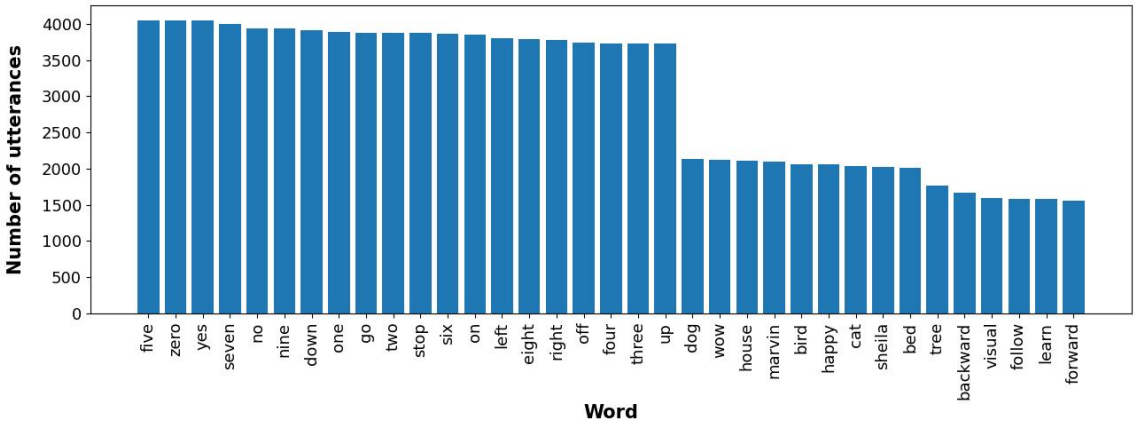

Figure 2.5: GSC utterances distribution

[1] `https://github.com/tensorflow/tensorflow/blob/master/tensorflow/examples/speech_commands/input_data.py`

[2] `http://download.tensorflow.org/data/speech_commands_v0.02.tar.gz`

```
GSC/
├── backward
│   ├── 0165e0e8_nohash_0.wav
│   ├── 017c4098_nohash_0.wav
│   └── ...
├── bed
│   ├── 00176480_nohash_0.wav
│   ├── 004ae714_nohash_0.wav
│   └── ...
└── ...
```

Figure 2.6: GSC files structure

2.3.2 MSWC

Multilingual Spoken Words Corpus [3] is a huge and expanding audio collection of spoken words in 50 languages, contributed by over 5 billion individuals. This dataset addresses the issues encountered in traditional counterparts, such as the time and effort required for collection, as well as enhances the richness and diversity of languages, speakers and environments in the dataset. MSWC is automatically generated by applying forced alignment, a technique that precisely aligns the spoken speech with its corresponding text transcription, at the level of individual words on Common Voice [1], a crowdsourced speech database of spoken sentences in multiple languages.

In total, the dataset contains approximately 22.5 million utterances of 340 thousand keywords (the distribution across 50 languages is shown in figure 2.7). The discrepancies in resource amount primarily stem from the popularity of each language and the participation rates of speakers. All of the utterances share the same 1-second (or less) duration to facilitate the standard usage in the KWS field. The audio files are stored in the OPUS format with the sampling rate of 48KHz, consistent with Common Voice. The dataset comes with prespecified train, dev, test splits for each keyword in the ratio of 8:1:1 with 2 guaranteed criteria: each split includes at least 1 utterance and no speakers appear in multiple splits for the same keyword.

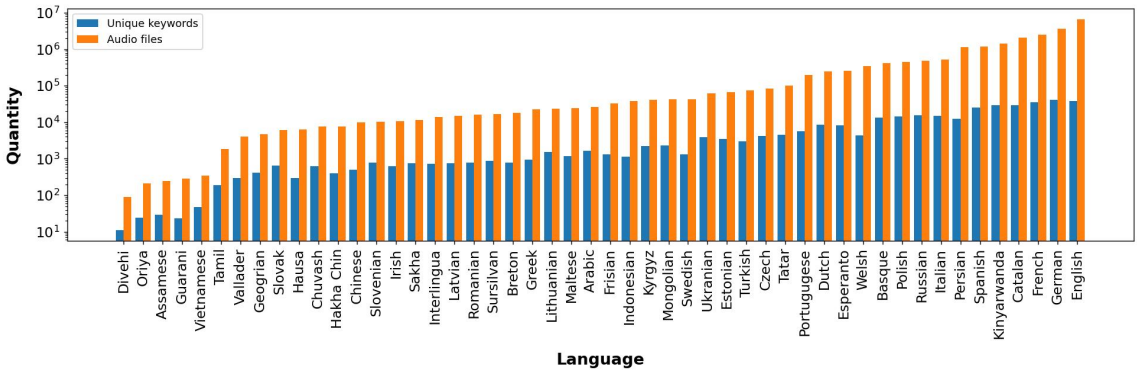

Figure 2.7: MSWC utterances distribution (log scaled)

```
MSWC/audio/
├─ vi/clips/
│  ├─ anh
│  │  ├─ common_voice_vi_21833244.opus
│  │  ├─ common_voice_vi_21833255.opus
│  │  └─ ...
│  ├─ cho
│  │  ├─ common_voice_vi_22028288.opus
│  │  ├─ common_voice_vi_22195135.opus
│  │  └─ ...
│  └─ ...
└─ ...
```

Figure 2.8: MSWC files structure

The dataset can be downloaded from this website [3] as a gzip-compressed tar archive, which occupies approximately 124 GiB of disk storage. After uncompressing, including nested gzip files, the audio files can be found within the *audio* folder, organized hierarchically from language to corresponding word (see figure 2.8). The splits index files are provided per language in the *splits* folder, prefixed with the language code for differentiation.

2.4 Evaluation Metrics

The evaluation of classification outputs in a keyword spotting (KWS) system involves two components: (i) binary classification, which distinguishes between keyword and non-keyword samples, and (ii) multi-class classification, which differentiates between individual keywords within the system.

[3] `https://mlcommons.org/datasets/multilingual-spoken-words/`

2.4.1 Confusion Matrix

A confusion matrix is a standard tool for visualizing the performance of a classification model by comparing predicted outcomes against ground-truth labels.

For binary classification (positive vs. negative classes), the confusion matrix consists of four elements:

- **True Positive (TP)**: Number of positive samples correctly identified as positive.
- **False Positive (FP)**: Number of negative samples incorrectly identified as positive.
- **True Negative (TN)**: Number of negative samples correctly identified as negative.
- **False Negative (FN)**: Number of positive samples incorrectly identified as negative.

In the multi-class case of KWS, the confusion matrix can be decomposed into multiple binary matrices by treating each class as positive while considering the others as negative. Standard binary metrics can then be computed per class and averaged (macro- or micro-averaging) to provide an overall performance measure.

True Positive Rate (TPR): The True Positive Rate is the proportion of correctly predicted positive instances among all actual positive instances:

$$\text{TPR} = \frac{TP}{TP+FN}.$$

False Positive Rate (FPR): The False Positive Rate is the proportion of incorrectly predicted positive instances among all actual negative instances:

$$\text{FPR} = \frac{FP}{FP+TN}.$$

2.4.2 AUC

The Area Under the ROC Curve (AUC) is a widely used metric for evaluating the overall performance of a binary classifier. The ROC curve plots the TPR against the FPR across different decision thresholds.

Since each classification is associated with a confidence score, the decision threshold determines the minimum score required to classify a sample as positive. A higher threshold

reduces both TPR and FPR (more conservative predictions), while a lower threshold increases both (more permissive predictions). Thus, the ROC curve provides a comprehensive view of the trade-off between sensitivity and specificity at different thresholds.

2.4.3 FAR and FRR

In KWS evaluations, two domain-specific metrics are often reported: the **False Acceptance Rate (FAR)** and the **False Rejection Rate (FRR)**.

- FAR measures how often the system incorrectly accepts non-keyword samples.
- FRR measures how often the system incorrectly rejects keyword samples.

Formally, FAR is equivalent to the FPR, while FRR corresponds to the False Negative Rate (FNR), defined as:

$$FAR = FPR, \quad FRR = \text{FNR} = 1 - TPR.$$

Similar to the ROC curve, the Detection Error Tradeoff (DET) curve visualizes system performance across thresholds, but plots FAR against FRR. Effectively, it provides the same information as the ROC curve, with the y-axis inverted (see Figure 2.9).

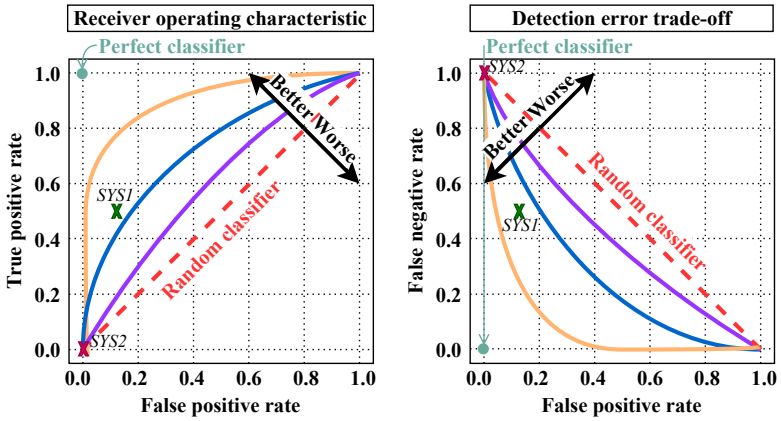

Figure 2.9: ROC vs DET curve comparision [2]

3. Methodology

The primary contribution of this study lies in providing an enhanced version of the testing protocol originally proposed by Rusci et al. [4] for few-shot open-set keyword spotting systems. Our improvements address three fundamental aspects of their evaluation framework: data utilization strategies, prediction score computation methods, and performance metrics selection. These enhancements focus exclusively on the enrollment and inference phases of the system, treating the feature encoder as a frozen function that transforms audio signals into embedding vectors, thereby maintaining consistency with the few-shot learning paradigm.

3.1 Analysis of the Original Approach

Before presenting our proposed improvements, we first provide a comprehensive analysis of the existing methodology to establish the foundation for our enhancements. Understanding the strengths and limitations of the current approach is essential for appreciating the necessity and impact of our proposed modifications.

3.1.1 Data Utilization Strategy

Rusci et al. [4] established a systematic approach to data partitioning using the Google Speech Commands (GSC) dataset. Their methodology involves creating two distinct partitions from the 35 available word classes. The positive partition, denoted as GSC+, comprises 10 target keyword classes (yes, no, up, down, left, right, on, off, stop, go), which represent the words that the system should recognize and classify correctly. The negative partition, GSC-, contains the remaining 20 word classes, with a notable exception of 5 specific words (backward, forward, visual, follow, learn) that are excluded entirely from the evaluation protocol. These excluded words serve a specialized purpose in computing

unknown prototypes when using the openNCM classifier architecture.

The original approach leverages the existing train-development-test split structure of the GSC dataset, which follows a hash function-based partitioning scheme applied to filenames, resulting in an 8:1:1 ratio across the three splits. For the enrollment phase, the system uses k-shot utterances randomly selected from the training split of GSC+, where k represents the number of examples per keyword class available during system setup. The inference and evaluation phases utilize all utterances from the test split of both GSC+ and GSC- partitions, creating a comprehensive evaluation scenario that includes both target keywords and unknown words.

3.1.2 Prediction Score Computation

The prediction score algorithm in the original framework operates through a multi-step distance computation and normalization process. When a query sample enters the system, the framework computes n L2 distances between the query embedding and the n target keyword prototypes. These prototypes are derived by averaging the embeddings of enrollment samples for each keyword class. The system then employs what the authors term ”open-set classifiers” (including openNCM, openMAX, and DProto variants) to generate an additional distance representing the query’s relationship to non-keyword content.

This additional distance computation introduces a virtual prototype representing all non-keywords, creating a total of n+1 distances. The system subsequently normalizes these distances to produce n+1 probability scores, which serve as the final prediction scores for classification decisions. This normalization step transforms the raw distance measurements into a probability distribution over all possible classes, including the unknown class.

3.1.3 Performance Metrics

The original evaluation protocol employs three complementary metrics to assess system performance across different operational aspects. The Area Under the ROC Curve (AUC) provides a comprehensive measure of the system’s overall ability to distinguish between target keywords and non-keywords across all possible threshold settings. This metric offers insight into the fundamental discriminative power of the learned representations.

The authors also introduce a threshold-based evaluation approach, where they fix the False Acceptance Rate (FAR) at 5 percent, meaning that among non-keyword utterances, only 5 percent are incorrectly accepted as valid keywords. At this operating point, they measure

two additional metrics: classification accuracy within the target keyword set (how well the system differentiates between different known keywords) and the False Rejection Rate (how frequently genuine keyword utterances are incorrectly classified as non-keywords).

3.2 Proposed Improvements

Building upon our analysis of the existing methodology, we propose several fundamental improvements that address key limitations while maintaining the core principles of few-shot open-set learning. These improvements aim to create a more realistic, generalizable, and interpretable evaluation framework.

3.2.1 Direct Distance-Based Classification

Our first major improvement involves eliminating the probability normalization step and utilizing L2 distances directly for classification decisions. This modification addresses several conceptual and practical limitations in the original approach while providing more intuitive threshold interpretation.

The fundamental issue with probability-based scoring lies in its dependency on the ”unknown” distance computation. Consider a scenario where we have five keyword prototypes, and a query sample produces distances of [1, 2, 3, 4, 5] to these prototypes respectively. If a second query sample generates distances of [2, 4, 6, 8, 10] (exactly double the first sample's distances), the normalized probability scores would be identical without the unknown distance computation. This mathematical artifact obscures important information about the absolute positioning of query samples in the embedding space.

The unknown distance serves as a normalization factor that helps differentiate between queries that are generally close to all prototypes versus those that are distant from all prototypes. However, this approach introduces artificial complexity and makes threshold interpretation less intuitive. In our direct distance approach, we set acceptance thresholds that can be understood as acceptance radii around each keyword prototype. A threshold calibrated to achieve 5 percent FAR can be interpreted as the radius within which we accept a query as belonging to a specific keyword class.

This geometric interpretation provides clearer insights into system behavior and removes the dependency on computing virtual non-keyword representations. Non-keywords are naturally handled through the rejection mechanism when queries fall outside all acceptance

radii, rather than requiring explicit prototype computation.

3.2.2 Mean Detection Error Tradeoff Curves

Our second major improvement replaces the three-metric evaluation approach with a comprehensive visualization method based on mean Detection Error Tradeoff (DET) curves. Traditional evaluation approaches using AUC, accuracy, and false rejection rate provide limited insight into system behavior across different operating conditions and fail to capture the complexity of multi-class keyword spotting performance.

The conventional ROC curve and its associated AUC metric focus exclusively on the binary discrimination between all keywords collectively versus all non-keywords. While this provides useful information about overall system performance, it fails to capture how well the system distinguishes between individual keyword classes, which is crucial for practical deployment scenarios.

Our approach computes individual DET curves for each target keyword class, treating each keyword against all other content (including both other keywords and non-keywords) as a separate binary classification problem. This methodology provides detailed insights into per-class performance while maintaining sensitivity to both false acceptance and false rejection characteristics.

The mean DET curve is obtained by averaging the individual DET curves across all target keyword classes, similar to the approach used in multi-class confusion matrix averaging. This averaging process preserves the detailed performance characteristics while providing a single, comprehensive performance summary that captures both keyword discrimination and rejection capabilities.

Each individual DET curve visualizes the tradeoff between false acceptance and false rejection for a specific keyword, revealing how well the system can distinguish that particular keyword from all other possible audio content. The resulting mean curve provides an overall performance assessment that naturally incorporates both the accuracy of keyword classification and the effectiveness of non-keyword rejection.

3.2.3 Enhanced Data Utilization Framework

Our third improvement addresses the lack of generalizability in the original data utilization strategy by introducing randomized word selection and expanding the evaluation dataset.

The original approach uses fixed word partitions across all evaluation runs, which limits the protocol's ability to simulate real-world deployment scenarios where users define arbitrary keyword sets.

We formalize our data utilization terminology to ensure clarity and reproducibility. The utterance splits used for enrollment and evaluation are termed the enrollment split and evaluation split respectively. The words selected as targets for the KWS system constitute the positive partition, while words chosen to represent non-keywords form the negative partition. This terminology provides a clear framework for describing different experimental configurations.

Our improved protocol introduces randomization in word selection for both positive and negative partitions across different evaluation runs. This randomization better simulates real-world scenarios where users can define arbitrary keyword sets, providing a more robust assessment of system generalizability. The variability in word selection also helps identify potential biases in the evaluation protocol that might arise from using fixed word sets.

3.2.4 Multi-Lingual Spoken Words Corpus Integration

To address the limited vocabulary size of the GSC dataset (35 words), we propose incorporating a subset of the Multi-lingual Spoken Words Corpus (MSWC) for evaluation purposes. The GSC dataset's small vocabulary limits the introduction of true randomness in word selection, potentially leading to overfitting of evaluation protocols to specific acoustic or linguistic characteristics.

Our MSWC subset selection criteria ensure both data quality and experimental validity. We include words with a minimum of 1000 utterances to guarantee sufficient data for robust evaluation, while excluding the first 500 words with the highest utterance counts, as these were used for training the feature encoder in the original study [4]. This exclusion prevents data leakage and ensures fair evaluation of few-shot learning capabilities. The resulting dataset contains 263 distinct words, providing substantially more vocabulary diversity than the original 35-word GSC set.

The utterance splitting strategy for MSWC differs significantly from GSC to optimize evaluation effectiveness. We employ a 1:9 ratio between enrollment and evaluation splits, contrasting with GSC's 8:1 ratio. This allocation strategy recognizes that few-shot learning scenarios require only k samples per keyword for enrollment, making large enrollment splits inefficient. The emphasis on evaluation data maximizes our ability to assess system

performance across diverse acoustic conditions and speaking styles.

For experimental configuration, we randomly select 5 words for the positive partition and 50 words for the negative partition from the 263 available words. The choice of 5 keywords reflects practical deployment scenarios where systems typically handle a small number of user-defined commands. The 1:10 ratio between positive and negative partition sizes provides sufficient negative examples for robust open-set evaluation while maintaining computational feasibility.

3.3 Experimental Design

We design four systematic experiments to validate our proposed improvements and demonstrate their effectiveness over existing approaches. Each experiment addresses a specific aspect of our methodology while building toward a comprehensive evaluation of the enhanced framework.

3.3.1 Direct Distance vs. Probability-Based Classification

This experiment provides a controlled comparison between our direct L2 distance approach and the probability-based scoring method originally proposed by Rusci et al. [4]. We replicate the best-performing system configuration reported in their work, maintaining identical feature encoders, training procedures, and datasets. The only modification is the replacement of probability normalization with a direct distance thresholding strategy. Furthermore, we evaluate the results using both conventional metrics for consistency with prior literature and the newly proposed metrics to demonstrate its improved interpretability.

This controlled design isolates the effect of our scoring modification from other experimental factors. We hypothesize that removing the artificial constraints introduced by the ”unknown” prototype and probability normalization will lead to superior performance. To ensure fairness, distance thresholds are calibrated to match equivalent operating points with the baseline system.

3.3.2 Data Utilization Strategy Comparison

This experiment compares three data utilization strategies arranged by increasing difficulty and real-world relevance. GSC Fixed uses the original predetermined word selections

across all runs, providing maximum control but limited generalizability. GSC Randomized maintains the same dataset but randomizes word selection for positive and negative partitions, introducing deployment-realistic variability. MSWC employs our proposed expanded vocabulary with randomized selection and optimized data splitting.

We hypothesize that difficulty increases sequentially across these strategies, with each approach revealing system limitations that simpler evaluations miss. This progressive analysis demonstrates that our improvements address genuine evaluation limitations rather than introducing artificial complexity. The experiment includes statistical analysis of performance distributions across randomization runs, examining both mean performance and variance characteristics.

3.3.3 Few-Shot Performance Scaling

The final experiment characterizes how system performance varies across different few-shot scenarios, examining the relationship between enrollment data availability and effectiveness. We systematically vary k from 1 to 20 shots per keyword, covering the practical few-shot learning spectrum from extremely limited to generous enrollment conditions.

Using our enhanced evaluation framework across all k-shot scenarios, we generate mean DET curves for each condition to reveal how overall performance and the balance between false acceptance and rejection evolve with data availability. The analysis includes comparative examination across all three data utilization strategies from Experiment 3, providing multi-dimensional insights into few-shot learning scaling characteristics and evaluation methodology sensitivity.

The experiment extends beyond accuracy measurements to examine stability and reliability, identifying minimum data requirements for stable performance and diminishing returns points where additional enrollment examples provide minimal improvements. This analysis bridges methodological validation with practical deployment guidance, offering actionable insights for optimizing enrollment burden versus system performance.

4. Results & Discussion

4.1 Direct Distance vs. Probability-Based Classification

Method	$AUC \uparrow$	$ACC^{+}_{5\%} \uparrow$	$FRR^{+}_{5\%} \downarrow$
Use probabilities	0.94	0.76	0.21
Use L2 distances	0.95	0.81	0.17

Table 4.1: Comparison of methods by AUC, acc@5, and frr@5.

Figure 4.1: Probability and L2 Distance Comparison

The experimental results in Table 4.1 demonstrate clear advantages of the direct L2 distance approach over probability-based classification. The improvement of 0.01 in AUC (0.95 vs 0.94) indicates better overall discriminative power, while the substantial increase in accuracy at 5% FAR (0.81 vs 0.76) and reduction in false rejection rate (0.17 vs 0.21) suggest more practical deployment benefits.

The DET curves in Figure 4.1 reveal that the direct distance method (GSC_dist) consistently outperforms the probability-based approach (GSC_prob) across all operating points. The orange curve representing direct distances lies below the blue probability curve throughout most of the false acceptance rate spectrum, indicating superior false rejection performance for any given false acceptance rate. This consistent advantage validates our hypothesis that removing artificial normalization constraints leads to more effective utilization of the embedding space geometry.

4.2 Data Utilization Strategy Comparison

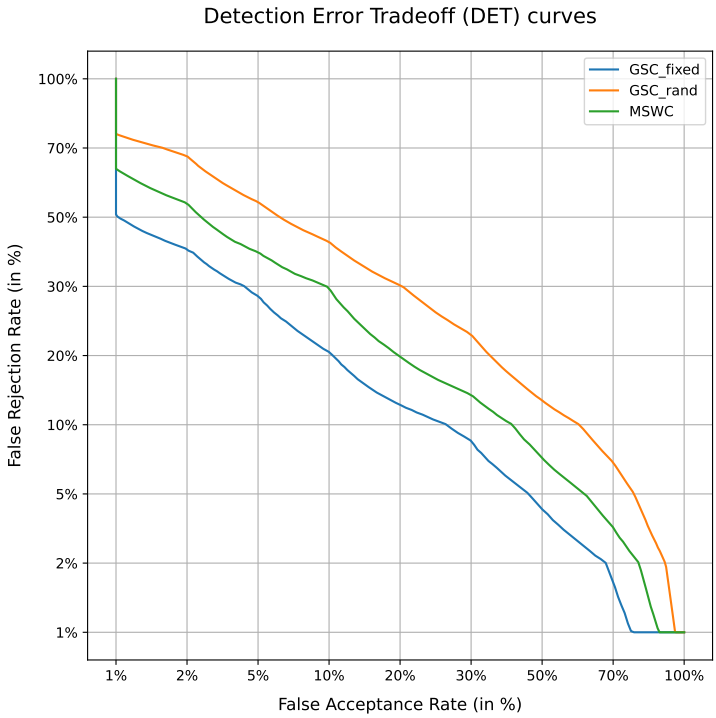

Figure 4.2: Data Utilization Comparison

Figure 4.2 demonstrates the expected hierarchy of evaluation difficulty across the three data utilization strategies. The GSC_Fixed configuration (blue curve) represents the easiest evaluation scenario, showing the best performance with false rejection rates reaching as low as 1% at moderate false acceptance rates. This reflects the artificial advantage gained from using predetermined word partitions that may favor specific acoustic or linguistic characteristics.

The GSC_Rand configuration (green curve) introduces realistic deployment variability through randomized word selection, resulting in moderately degraded performance compared to fixed partitions. The performance gap between fixed and randomized GSC evaluation reveals the extent to which fixed word selections can provide overly optimistic performance estimates.

The MSWC_Rand configuration (orange curve) presents the most challenging evaluation scenario, with significantly higher false rejection rates across all operating points. This increased difficulty stems from the expanded vocabulary diversity and the optimized 1:9 enrollment-to-evaluation split ratio. The MSWC results likely provide the most realistic assessment of system performance in practical deployment scenarios where users define arbitrary keyword sets from diverse linguistic backgrounds.

The substantial performance differences between evaluation strategies highlight the importance of realistic testing protocols. Systems that appear highly effective under fixed GSC evaluation may struggle significantly in real-world deployment conditions, as captured by the MSWC evaluation framework.

4.3 Few-Shot Performance Scaling

Figure 4.3 reveals critical insights into the relationship between enrollment data availability and system effectiveness. The curves show clear performance improvements as the number of enrollment samples increases from 1 to 10 shots, with diminishing returns becoming apparent beyond 5 shots.

The 1-shot scenario (blue curve) exhibits the poorest performance, with high false rejection rates across all false acceptance levels. This reflects the inherent challenge of learning reliable keyword representations from single examples. The substantial improvement observed when moving to 3-shot (orange curve) suggests that minimal additional enrollment data provides significant benefits.

The convergence of 5-shot (green) and 10-shot (red) curves indicates that performance

gains plateau relatively quickly in few-shot scenarios. Beyond 5 enrollment samples per keyword, additional data provides marginal improvements, suggesting an optimal balance between enrollment burden and system performance.

This scaling behavior has practical implications for deployment strategies. While 1-shot scenarios remain challenging, 3-5 shot enrollment appears to capture most achievable performance benefits without imposing excessive data collection burden on users. The rapid performance saturation also validates the few-shot learning paradigm for KWS applications, demonstrating that effective systems can be built with minimal user-provided data.

The analysis supports the hypothesis that few-shot learning approaches provide practical solutions for customizable keyword spotting, with reasonable performance achievable through modest enrollment requirements that users can realistically provide.

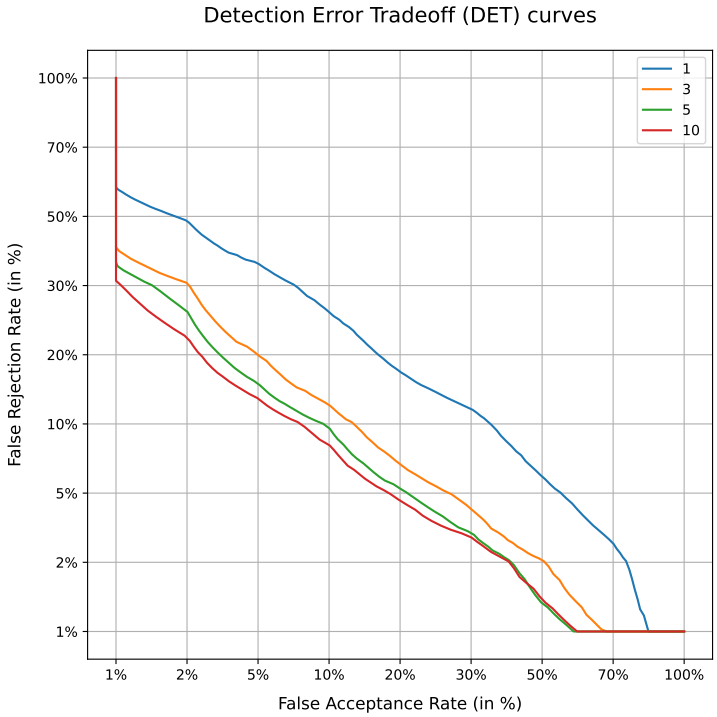

Figure 4.3: Different number of enrollment samples comparison

5. Conclusions & Future works

This study introduces an enhanced evaluation framework for few-shot open-set keyword spotting systems that addresses critical limitations in existing testing protocols. Through systematic experimental validation, we demonstrate three key methodological improvements that provide more rigorous and realistic assessment of system performance.

Our direct L2 distance-based classification approach eliminates the artificial constraints introduced by probability normalization and virtual prototype computation, resulting in measurable performance improvements and more intuitive threshold interpretation. The geometric understanding of acceptance radii around keyword prototypes provides clearer insights into system behavior compared to probability-based scoring methods.

The comprehensive evaluation using mean Detection Error Tradeoff curves reveals per-class performance characteristics that conventional metrics fail to capture. This approach naturally incorporates both keyword discrimination accuracy and unknown rejection effectiveness, providing a more complete assessment of open-set classification performance across different operating conditions.

The enhanced data utilization framework, incorporating randomized word selection and expanded vocabulary through MSWC integration, exposes significant performance variations that fixed evaluation protocols mask. Our results demonstrate that systems appearing highly effective under controlled conditions may struggle considerably in realistic deployment scenarios, highlighting the importance of rigorous evaluation methodologies.

The few-shot performance scaling analysis reveals that practical systems can achieve reasonable performance with 3-5 enrollment samples per keyword, with diminishing returns beyond this range. This finding validates the few-shot learning paradigm for keyword spotting applications while providing actionable guidance for balancing enrollment burden against system effectiveness.

5.1 Future Works

Multi-Modal Evaluation Frameworks: The current evaluation focuses on audio-only keyword spotting, but real-world deployment scenarios often involve multi-modal contexts. Extending the evaluation framework to incorporate visual cues, environmental context, or user behavior patterns could provide more comprehensive system assessment and reveal additional performance limitations.

Computational Efficiency Analysis: While this study focuses on classification accuracy and evaluation methodology, practical deployment requires consideration of computational constraints. Future work should integrate efficiency metrics into the evaluation framework, examining the tradeoffs between performance improvements and computational overhead, particularly for edge device deployment.

Personalization and Adaptation: The current framework treats all users equivalently, but real-world systems may benefit from user-specific adaptations. Investigating how the evaluation methodology can assess personalization capabilities and long-term adaptation performance could provide valuable insights for practical deployment scenarios.

Cross-Language Generalization: Although we introduce MSWC for expanded vocabulary evaluation, systematic investigation of cross-language generalization capabilities remains unexplored. Future studies should examine how systems trained on specific language combinations perform when deployed in multilingual environments, potentially revealing important generalization limitations.

Integration with Real-Time Processing: The current methodology focuses on offline evaluation, but practical systems require real-time processing capabilities. Future work should extend the evaluation framework to assess performance under real-time constraints, including latency requirements and streaming audio processing challenges.

These research directions collectively aim to bridge the gap between laboratory evaluation and practical deployment, ensuring that few-shot open-set keyword spotting systems can reliably serve users in diverse real-world conditions while maintaining the rigorous evaluation standards established in this work.

References

[1] Rosana Ardila et al. “Common voice: A massively-multilingual speech corpus”. In: *arXiv preprint arXiv:1912.06670* (2019).

[2] Iván López-Espejo et al. “Deep spoken keyword spotting: An overview”. In: *IEEE Access* 10 (2021), pp. 4169–4199.

[3] Mark Mazumder et al. “Multilingual spoken words corpus”. In: *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*. 2021.

[4] Manuele Rusci and Tinne Tuytelaars. “Few-Shot Open-Set Learning for On-Device Customization of KeyWord Spotting Systems”. In: *arXiv preprint arXiv:2306.02161* (2023).

[5] Jake Snell, Kevin Swersky, and Richard Zemel. “Prototypical networks for few-shot learning”. In: *Advances in neural information processing systems* 30 (2017).

[6] Pete Warden. “Speech commands: A dataset for limited-vocabulary speech recognition”. In: *arXiv preprint arXiv:1804.03209* (2018).

[7] Yundong Zhang et al. “Hello edge: Keyword spotting on microcontrollers”. In: *arXiv preprint arXiv:1711.07128* (2017).